

LAPORAN PRAKTIKUM STRUKTUR DATA DAN ALGORITMA

Analisis Kompleksitas Algoritma



DI SUSUN OLEH :

NAMA	: FARA MUTIA
NIM	: 2025573010034
PRODI	: TEKNIK INFORMATIKA
DOSEN PENGAJAR	: M. REZA ZULMAN, S.ST.,M.Sc

JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER

POLITEKNIK NEGERI LHOKSEUMAWE

LHOKSEUMAWE

2026

LEMBARAN PENGESAHAN

MATA KULIAH : Praktikum Struktur Data dan Algoritma
JUDUL PRAKTIKUM : Analisis Kompleksitas Algoritma
HARI PRAKTIKUM : Kamis
TANGGAL PENYERAHAN : 23 April 2026
NAMA : Fara Mutia
NIM : 2025573010034
PRODI/KELAS : Teknik Informatika/I D
NILAI :

Mahasiswa

Bukit Rata, 25 April 2026
Dosen Pembimbing

Fara Mutia
NIM. 2025573010034

M. REZA ZULMAN S.ST.,M.Sc
NIP. 19920501 2022031005

KATA PENGANTAR

Puji syukur saya panjatkan ke hadirat Tuhan Yang Maha Esa, karena berkat rahmat dan karunia-Nya, laporan ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai bagian dari tugas dalam mata kuliah Praktikum Struktur Data dan Algoritma yang bertujuan untuk memberikan pemahaman tentang Analisis Kompleksitas Algoritma.

Saya menyadari bahwa laporan ini tidak akan terwujud tanpa bantuan dan dukungan dari berbagai pihak. Oleh karena itu, saya mengucapkan terima kasih yang sebesar-besarnya kepada dosen pembimbing, teman-teman, serta pihak-pihak yang telah memberikan bantuan baik secara langsung maupun tidak langsung dalam penyelesaian laporan ini.

Semoga laporan ini dapat memberikan manfaat dan menambah wawasan bagi semua pihak yang membacanya. Kritik dan saran yang konstruktif sangat diharapkan untuk penyempurnaan laporan ini di masa mendatang.

Bukit Rata, 25 April 2026
Penulis

Fara Mutia
NIM. 2025573010034

DAFTAR ISI

LEMBARAN PENGESAHAN	ii
KATA PENGANTAR	iii
DAFTAR ISI	iv
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum	1
BAB II LANDASAN TEORI	2
2.1 Time Complexity (Kompleksitas Waktu)	2
2.2 Space Complexity (Kompleksitas Ruang)	2
2.3 Big O Notation	3
2.4 Analisis Struktur Perulangan (Loop).....	3
2.5 Time-Space Trade-off	3
BAB III METOLOGI PERCOBAAN	4
3.1 Mempersiapkan Project Praktikum	4
3.2 Mengerjakan Contoh, Latihan dan Tugas	4
3.3 Sumbit ke GitHub	14
BAB IV PEMBAHASAN DAN ANALISIS	16
4.1 PEMBAHASAN	16
4.2 ANALISA	16
BAB V PENUTUP.....	18
5.1 KESIMPULAN.....	18
BAB VI DAFTAR PUSAKA.....	19

BAB I PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman dan pengembangan perangkat lunak, efisiensi algoritma merupakan hal yang sangat penting. Seiring dengan meningkatnya jumlah data yang harus diproses, algoritma yang tidak efisien dapat menyebabkan waktu eksekusi yang sangat lama bahkan tidak praktis untuk digunakan. Oleh karena itu, diperlukan suatu metode untuk mengukur dan menganalisis performa algoritma, yaitu melalui analisis kompleksitas algoritma.

Analisis kompleksitas algoritma berfokus pada bagaimana waktu eksekusi (time complexity) dan penggunaan memori (space complexity) meningkat seiring dengan bertambahnya ukuran input. Salah satu pendekatan yang digunakan adalah notasi Big O, yang menggambarkan laju pertumbuhan dari suatu algoritma.

Dalam praktikum ini, mahasiswa mempelajari konsep dasar analisis kompleksitas algoritma menggunakan JavaScript dan Node.js. Pemahaman ini menjadi fondasi penting sebelum mempelajari struktur data yang lebih kompleks, karena tanpa memahami efisiensi algoritma, sulit untuk menentukan apakah suatu solusi pemrograman sudah optimal atau belum.

1.2 Tujuan Praktikum

Setelah menyelesaikan praktikum ini, diharapkan mampu:

1. Mengetahui konsep Time Complexity dan Space Complexity
2. Mengerti notasi Big O: $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$
3. Mampu menganalisis kompleksitas loop tunggal dan nested loop
4. Mampu membandingkan efisiensi dua algoritma secara empiris menggunakan `Date.now()`
5. Mampu mengenali pola kode yang memiliki kompleksitas buruk dan cara memperbaikinya

BAB II LANDASAN TEORI

2.1 Time Complexity (Kompleksitas Waktu)

Time complexity adalah ukuran yang menunjukkan seberapa lama waktu yang dibutuhkan oleh suatu algoritma untuk menyelesaikan masalah berdasarkan ukuran input (n). Kompleksitas waktu tidak diukur dalam satuan detik, melainkan dalam jumlah langkah atau operasi yang dilakukan oleh algoritma.

Notasi yang digunakan untuk menggambarkan time complexity adalah Big O Notation, yang menyatakan laju pertumbuhan waktu eksekusi terhadap ukuran input. Beberapa kelas kompleksitas yang umum digunakan antara lain:

- $O(1)$ (Konstan): Waktu eksekusi tidak bergantung pada ukuran input. Contohnya adalah akses elemen array berdasarkan indeks.
- $O(\log n)$ (Logaritmik): Waktu eksekusi bertambah secara logaritmik. Biasanya terjadi pada algoritma yang membagi masalah menjadi dua bagian, seperti binary search.
- $O(n)$ (Linear): Waktu eksekusi bertambah sebanding dengan jumlah input, seperti pada perulangan tunggal.
- $O(n \log n)$: Kombinasi antara linear dan logaritmik, sering ditemukan pada algoritma pengurutan efisien seperti merge sort.
- $O(n^2)$ (Kuadratik): Terjadi pada algoritma dengan nested loop, di mana jumlah operasi meningkat sangat cepat.
- $O(2^n)$ (Eksponensial): Kompleksitas yang sangat tinggi dan tidak efisien untuk input besar, biasanya terjadi pada algoritma rekursif tanpa optimasi.

Semakin kecil nilai kompleksitas, maka semakin efisien algoritma tersebut dalam menangani data besar.

2.2 Space Complexity (Kompleksitas Ruang)

Selain waktu, penggunaan memori juga menjadi faktor penting dalam menentukan efisiensi algoritma. Space complexity mengukur jumlah memori tambahan (auxiliary memory) yang digunakan selama proses eksekusi algoritma. Beberapa kategori space complexity antara lain:

- $O(1)$: Menggunakan memori tambahan yang tetap, tidak bergantung pada ukuran input.
- $O(n)$: Membutuhkan memori tambahan yang sebanding dengan jumlah input, misalnya saat membuat array baru.
- $O(n)$ pada rekursi: Terjadi karena penggunaan call stack yang bertambah seiring jumlah pemanggilan fungsi.

Konsep ini penting karena dalam beberapa kasus, penggunaan memori yang terlalu besar dapat menyebabkan program menjadi tidak efisien atau bahkan gagal dijalankan.

2.3 Big O Notation

Big O Notation adalah cara untuk menyatakan batas atas (upper bound) dari pertumbuhan kompleksitas algoritma. Notasi ini membantu dalam membandingkan efisiensi beberapa algoritma yang berbeda. Terdapat beberapa aturan dalam penyederhanaan Big O, yaitu:

1. Mengabaikan konstanta, misalnya $O(2n)$ disederhanakan menjadi $O(n)$.
2. Mengambil suku dengan pertumbuhan paling besar, misalnya $O(n^2 + n)$ menjadi $O(n^2)$.
3. Fokus pada pertumbuhan saat n sangat besar ($n \rightarrow \infty$).

Dengan aturan ini, analisis menjadi lebih sederhana dan mudah dibandingkan antar algoritma.

2.4 Analisis Struktur Perulangan (Loop)

Struktur perulangan dalam kode sangat mempengaruhi kompleksitas algoritma. Beberapa pola umum antara lain:

- Perulangan tunggal (single loop) biasanya memiliki kompleksitas $O(n)$.
- Perulangan bersarang (nested loop) memiliki kompleksitas $O(n^2)$ atau lebih tinggi tergantung jumlah lapisan.
- Perulangan dengan pembagian dua ($i \div 2$) menghasilkan kompleksitas $O(\log n)$.

Dengan mengenali pola ini, seorang programmer dapat dengan cepat mengidentifikasi kompleksitas suatu algoritma hanya dengan melihat struktur kodenya.

2.5 Time-Space Trade-off

Dalam pengembangan algoritma, sering terjadi trade-off antara waktu dan ruang. Artinya, untuk membuat algoritma lebih cepat, kadang diperlukan penggunaan memori yang lebih besar, dan sebaliknya.

Contoh umum adalah penggunaan teknik memoization pada algoritma rekursif. Teknik ini dapat mengurangi waktu eksekusi secara signifikan dari $O(2^n)$ menjadi $O(n)$, tetapi membutuhkan tambahan memori untuk menyimpan hasil perhitungan sebelumnya.

Pemahaman terhadap trade-off ini sangat penting agar dapat memilih solusi yang paling sesuai dengan kebutuhan sistem.

BAB III METOLOGI PERCOBAAN

3.1 Mempersiapkan Project Praktikum

1. Buka App Git Bash. Kemudian navigasikan ke root folder repository.

```
newuser@DESKTOP-JP3BR80 MINGW64 ~  
$ cd '/e/Praktikum Struktur Data/202573010034_strukturdatadanaalgoritma'
```

2. Buat folder baru dengan nama praktikum-5 untuk menyimpan semua file pada pertemuan ini dan masuk kedalamnya.

```
newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur  
datadanaalgoritma (main)  
$ mkdir praktikum-5  
  
newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur  
datadanaalgoritma (main)  
$ cd praktikum-5
```

3. Inisialisasi npm project.

```
newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur  
datadanaalgoritma/praktikum-5 (main)  
$ npm init -y  
Wrote to E:\Praktikum Struktur Data\202573010034_strukturdatadanaalgoritma\prakti  
kum-5\package.json:  
  
{  
  "name": "praktikum-5",  
  "version": "1.0.0",  
  "description": "",  
  "main": "index.js",  
  "scripts": {  
    "test": "echo \"Error: no test specified\" && exit 1"  
  },  
  "keywords": [],  
  "author": "",  
  "license": "ISC",  
  "type": "commonjs"  
}
```

3.2 Mengerjakan Contoh, Latihan dan Tugas

3.2.1 Mengukur Waktu Eksekusi Empiris

1. Membuat file baru dengan nama 01-intro-kompleksitas.js di dalam folder praktikum-5

```
newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur  
datadanaalgoritma/praktikum-5 (main)  
$ touch 01-intro-kompleksitas.js
```

2. Lalu buka file tersebut di VS Code. Ketik contoh codenya. Simpan file, lalu jalankan untuk melihat hasil outputnya ketik **01-intro-kompleksitas.js** di terminal.


```

1 function ukurWaktu(label, fn){
2   const mulai = Date.now();
3   fn();
4   const selesai = Date.now();
5   console.log(`${label}: ${selesai - mulai} ms`);
6 }
7 const N = 100_000;
8
9 function jumlahkanLinear(n){
10  let total = 0;
11  for(let i = 1; i <= N; i++) total += i;
12  return total;
13 }
14
15 function jumlahkanRumus(n){
16  return (n * (n + 1)) / 2;
17 }
18
19 function cariPasangan(arr){
20  const pasangan = [];
21  for (let i = 0; i < arr.length; i++){
22    for (let j = i + 1; j < arr.length; j++){
23      if (arr[i] + arr[j] === 10) pasangan.push([arr[i], arr[j]]);
24    }
25  }
26  return pasangan;
27 }
28
29 const data = Array.from({length: 5000}, (_, i) => i);
30
31 console.log('=== Perbandingan Waktu Eksekusi ===');
32 ukurWaktu('O(1) jumlahkanRumus ', () => jumlahkanRumus(N));
33 ukurWaktu('O(n) jumlahkanLinear ', () => jumlahkanLinear(N));
34 ukurWaktu('O(n^2) cariPasangan ', () => cariPasangan(data));
35
36 console.log('\nHasil sama?', jumlahkanLinear(100) === jumlahkanRumus(100));
37

```

PS E:\Praktikum Struktur Data\202573010034_strukturdatadanaalgoritma\praktikum-5> node .\01-intro-kompleksitas.js

=== Perbandingan Waktu Eksekusi ===

O(1) jumlahkanRumus : 0 ms

O(n) jumlahkanLinear : 2 ms

O(n²) cariPasangan : 41 ms

Hasil sama? false

PS E:\Praktikum Struktur Data\202573010034_strukturdatadanaalgoritma\praktikum-5>

2.2.2 Notasi Big O

1. Membuat file baru dengan nama 02-kelas-big-o.js di dalam folder praktikum-5

```

newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_strukturdatadanaalgoritma/praktikum-5 (main)
$ touch 02-kelas-big-o.js

```

2. Lalu buka file tersebut di VS Code. Ketik contoh codenya.

```

1 function ambilPertama(arr) { return arr[0]; }
2 function tambahkanItem(arr, item) { arr.push(item); }
3 function isGenap(n) { return n % 2 === 0; }
4
5 function binarySearch(arr, target){
6   let kiri = 0, kanan = arr.length - 1;
7   let langkah = 0;
8   while (kiri <= kanan){
9     langkah++;
10    const tengah = Math.floor((kiri + kanan) / 2);
11    if (arr[tengah] === target){
12      console.log(` Ditemukan di indeks ${tengah} setelah ${langkah} langkah`);
13      return tengah;
14    }
15    if (arr[tengah] < target) kiri = tengah + 1;
16    else kanan = tengah - 1;
17  }
18  return -1;
19 }
20
21 function cariMax(arr) {
22   let maks = arr[0];
23   for (const x of arr) if (x > maks) maks = x;
24   return maks;
25 }
26

```

```

27 function bubbleSortNaif(arr) {
28   const a = [...arr];
29   for (let i = 0; i < a.length; i++)
30     for (let j = 0; j < a.length - i - 1; j++)
31       if (a[j] > a[j+1]) [a[j], a[j+1]] = [a[j+1], a[j]];
32   return a;
33 }
34
35 function fibRekursif(n) {
36   if (n <= 1) return n;
37   return fibRekursif(n-1) + fibRekursif(n-2);
38 }
39 console.log('=== O(1) - selalu cepat ===');
40 console.log(ambilPertama([10,20,30,40,50]));
41 console.log(isGenap(42));
42
43 console.log('\n=== O(log n) - Binary Search ===');
44 const sorted = Array.from({length:1_000_000}, (_,i)=>i);
45 binarySearch(sorted, 731_452);
46
47 console.log('\n=== O(n) - Linear Search ===');
48 console.log('Max dari 1000 elemen:',
49 cariMax(Array.from({length:1000}, ()=>Math.random()*1000|0)));
50
51 console.log('\n=== O(n^2) - Bubble Sort (kecil saja) ===');
52 console.log(bubbleSortNaif([64, 34, 25, 12, 22, 11, 90]));
53
54 console.log('\n=== O(2^n) - Fibonacci Rekursif (n kecil!) ===');
55 for (let i = 0; i <= 10; i++) process.stdout.write(fibRekursif(i) + ' ');
56 console.log('');
57
58 console.log('\nWaktu fib(35) O(2^n):');
59 let t = Date.now(); fibRekursif(35); console.log(Date.now()-t, 'ms');
60 console.log('Waktu fib(35) O(n) memoization:');
61 const memo = {};
62 function fibMemo(n) {
63   if (n <= 1) return n;
64   if (memo[n]) return memo[n];
65   return memo[n] = fibMemo(n-1) + fibMemo(n-2);
66 }
67 t = Date.now(); fibMemo(35); console.log(Date.now()-t, 'ms');

```

3. Simpan file, lalu jalankan untuk melihat hasil outputnya ketik **node 02-kelas-big-o.js** di terminal.

```

PS E:\Praktikum Struktur Data\202573010034_strukturdata
n\algoritma\praktikum-5> node .\02-kelas-big-o.js
=== O(1) - selalu cepat ===
10
true

=== O(log n) - Binary Search ===
Ditemukan di indeks 731452 setelah 19 langkah

=== O(n) - Linear Search ===
Max dari 1000 elemen: 999

=== O(n^2) - Bubble Sort (kecil saja) ===
[
  11, 12, 22, 25,
  34, 64, 90
]

=== O(2^n) - Fibonacci Rekursif (n kecil!) ===
0 1 1 2 3 5 8 13 21 34 55

Waktu fib(35) O(2^n):
184 ms
Waktu fib(35) O(n) memoization:
0 ms

```

4. Buat file baru dengan nama latihan-1.js

```
newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur
datadana/goritma/praktikum-5 (main)
$ touch latihan-1.js
```

5. Buka file di VS Code dan kerjakan latihannya.

```
1  function fnA(n){
2      return n*2;
3  }
4  // Big O: O(1) - Alasan: Hanya satu operasi aritmatika tunggal.
5
6  function fnB(n) {
7      for (let i=0; i<n; i++){
8          for (let j=0; j<n; j++){
9              //console.log(i,j);
10             }
11         }
12     }
13     // Big O: O(n^2) - Alasan: Loop bersarang dua tingkat (n * n).
14
15     function fnC(n){
16         for (let i=1; i<n; i*=2) {
17             //console.log(i);
18         }
19     }
20     // Big O: O(log n)
21
22     function fnD(n){
23         let arr = Array.from({ length: n }, (_, i) => i);
24
25         arr.forEach(x => {
26             arr.forEach(y => {
27                 arr.forEach(z => {
28                     //console.log(x, y, z);
29                 });
30             });
31         });
32     }
33     // Big O: O(n^3) - Alasan: Loop bersarang tiga tingkat (n * n * n).
34
35
36     function hitungKompleksitas(n, fn){
37         const start = Date.now();
38         fn(n);
39
40         const end = Date.now();
41         const waktu = end - start;
42
43         console.log(`Waktu eksekusi untuk n=${n}: ${waktu} ms`);
44     }
45
46     const N = 1000;
47     console.log("=== Fungsi A : O(1) ===");
48     hitungKompleksitas(N, fnA);
49     console.log("=== Fungsi B : O(n^2) ===");
50     hitungKompleksitas(N, fnB);
51     console.log("=== Fungsi C O(log n)===");
52     hitungKompleksitas(N, fnC);
53     console.log("=== Fungsi D : O(n^3)===");
54     hitungKompleksitas(N, fnD);
```

6. Simpan dan jalankan file di terminal

```
PS E:\Praktikum Struktur Data\202573010034_strukturdatadanalgori
tma\praktikum-5> node .\latihan-1.js
=== Fungsi A : O(1) ===
Waktu eksekusi untuk n=1000: 0 ms
=== Fungsi B : O(n^2) ===
Waktu eksekusi untuk n=1000: 4 ms
=== Fungsi C O(log n)===
Waktu eksekusi untuk n=1000: 0 ms
=== Fungsi D : O(n^3)===
Waktu eksekusi untuk n=1000: 1053 ms
```

2.2.3 Space Complexity

1. Buat file baru dengan nama 03-space-complexity.js di dalam folder praktikum-5.

```
newuser@DESKTOP-JP3BRBO MINGW64 /e/Praktikum Struktur Data/202573010034_struktur
datadanalgoritma/praktikum-5 (main)
$ touch 03-space-complexity.js
```

2. Lalu buka file tersebut di VS Code. Ketik contoh codenya dan kerjakan latihannya.

```
1  function jumlahArray(arr){
2      let total = 0;
3      for (const x of arr) total += x;
4      return total;
5  }
6
7  function duplikasiArray(arr) {
8      const baru = [];
9      for (const x of arr) baru.push(x * 2);
10     return baru;
11 }
12
13
14 function faktorialRekursif(n) {
15     if (n <= 1) return 1;
16     return n * faktorialRekursif(n - 1);
17 }
18
19 function faktorialIteratif(n) {
20     let hasil = 1;
21     for (let i = 2; i <= n; i++) hasil *= i;
22     return hasil;
23 }
24
25 const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
26
27 console.log('Jumlah array   :', jumlahArray(arr));
28 console.log('Duplikasi array:', duplikasiArray(arr));
29 console.log('Faktorial 10 rekursif :', faktorialRekursif(10));
30 console.log('Faktorial 10 iteratif :', faktorialIteratif(10));
31
32 function hitungUnik(arr){
33     const seen = new Set();
34     for (const x of arr) seen.add(x);
35     return seen.size;
36 }
37 const dataAcak = [1,2,3,2,1,4,5,3,6,4];
38 console.log('Elemen unik:', hitungUnik(dataAcak));
```

```

40 //LATIHAN
41 console.log('\nlatihan ');
42 function cariPasanganLambat(arr, target){
43     for (let i = 0; i < arr.length; i++){
44         for(let j = i + 1; j < arr.length; j++){
45             if (arr[i] + arr[j] === target){
46                 return [arr[i], arr[j]];
47             }
48         }
49     }
50     return null;
51 }
52
53 function cariPasanganCepat(arr, target){
54     const set = new Set();
55     for(let num of arr){
56         const pasangan = target - num;
57         if(set.has(pasangan)){
58             return [pasangan, num];
59         }
60         set.add(num);
61     }
62     return null;
63 }
64
65 const data = [2,7,11,15];
66 const target = 9;
67
68 console.log('lambat :', cariPasanganLambat(data, target));
69 console.log('cepat :', cariPasanganCepat(data, target));
70
71 function ukurWaktu(fn, arr, target){
72     const start = Date.now();
73     fn(arr, target);
74     const end = Date.now();
75     return end - start;
76 }
77
78 const arrBesar = Array.from({length: 5000}, () => Math.floor(Math.random() * 100000));
79 const targetBesar = -1;
80
81 console.log('lambat: ', ukurWaktu(cariPasanganLambat, arrBesar, targetBesar), 'ms');
82 console.log('cepat: ', ukurWaktu(cariPasanganCepat, arrBesar, targetBesar), 'ms');

```

3. Simpan file, lalu jalankan untuk melihat hasil outputnya ketik **node 03-space-complexity.js** di terminal.

```

PS E:\Praktikum Struktur Data\202573010034_strukturdatadanal
goritma\praktikum-5> node .\03-space-complexity.js
Jumlah array : 55
Duplikasi array: [
  2, 4, 6, 8, 10,
  12, 14, 16, 18, 20
]
Faktorial 10 rekursif : 3628800
Faktorial 10 iteratif : 3628800
Elemen unik: 6

latihan
lambat : [ 2, 7 ]
cepat : [ 2, 7 ]
lambat: 36 ms
cepat: 2 ms

```

2.2.4 Mengenali Pola Kompleksitas

1. Membuat file baru dengan nama 04-refactor.js di dalam folder praktikum-5.

```
newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur
datadanaalgoritma/praktikum-5 (main)
$ touch 04-refactor.js
```

2. Lalu buka file tersebut di VS Code. Ketik contoh codenya. Simpan file, lalu jalankan js di terminal.

```
1 function adaDuplikatLambat(arr){
2   for(let i = 0; i < arr.length; i++)
3     for(let j = i+1; j < arr.length; j++)
4       if(arr[i] === arr[j]) return true;
5   return false;
6 }
7
8 function adaDuplikatCepat(arr){
9   const seen = new Set();
10  for(const x of arr){
11    if(seen.has(x)) return true;
12    seen.add(x);
13  }
14  return false;
15 }
16
17 function frekuensiLambat(arr) {
18   const unik = [];
19   const hitung = [];
20   for (const x of arr) {
21     const idx = unik.indexOf(x);
22     if (idx === -1) { unik.push(x); hitung.push(1); }
23     else hitung[idx]++;
24   }
25   return Object.fromEntries(unik.map((u,i) => [u, hitung[i]]));
26 }
27
28 function frekuensiCepat(arr){
29   const counter = {};
30   for (const x of arr) counter[x] = (counter[x] || 0) + 1;
31   return counter;
32 }
33
34 const besar = Array.from({length: 20000}, () => Math.floor(Math.random() * 1000));
35
36 let t = Date.now();
37 adaDuplikatLambat(besar);
38 console.log('Duplikat LAMBAT O(n²):', Date.now()-t, 'ms');
39
40 t = Date.now();
41 adaDuplikatCepat(besar);
42 console.log('Duplikat CEPAT O(n) :', Date.now()-t, 'ms');
43
44 const kata = ['a', 'b', 'a', 'c', 'b', 'a', 'd'];
45 console.log('\nFrekuensi:', frekuensiCepat(kata));
46
```

```
PS E:\Praktikum Struktur Data\202573010034_strukturdatadanaalgorit
ma\praktikum-5> node .\04-refactor.js
Duplikat LAMBAT O(n²): 0 ms
Duplikat CEPAT O(n) : 0 ms

Frekuensi: { a: 3, b: 2, c: 1, d: 1 }
PS E:\Praktikum Struktur Data\202573010034_strukturdatadanaalgorit
ma\praktikum-5>
```

2.2.5 Tugas Praktikum

1. Buat folder tugas/

```
newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur
datadanaalgoritma/praktikum-5 (main)
$ mkdir tugas
```

2. Lalu tambahkan dua file tugas di dalam folder tugas/ tersebut yaitu tugas-1.js dan tugas-2.js.

```
newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_struktur
datadana1goritma/praktikum-5 (main)
$ touch tugas/tugas-1.js tugas/tugas-2.js
```

3. Program tugas 1 beserta hasil outputnya.

```
1 //Fungsi A
2 //Versi  $O(n^2)$  | Alasan: nested loop (setiap elemen arr1 dibandingkan ke arr2)
3 function intersectionOn2(arr1, arr2) {
4     let result = [];
5
6     for (let i = 0; i < arr1.length; i++) {
7         for (let j = 0; j < arr2.length; j++) {
8             if (arr1[i] === arr2[j]) {
9                 if (!result.includes(arr1[i])) {
10                     result.push(arr1[i]);
11                 }
12             }
13         }
14     }
15
16     return result;
17 }
18 // Time:  $O(n^2)$ , Space:  $O(n)$ 
19
20 // Versi  $O(n)$  menggunakan Set | Alasan: akses Set  $O(1)$ , hanya 1 Loop utama
21 function intersectionOn(arr1, arr2) {
22     let set2 = new Set(arr2);
23     let result = new Set();
24
25     for (let item of arr1) {
26         if (set2.has(item)) {
27             result.add(item);
28         }
29     }
30     return [...result];
31 }
32 // Time:  $O(n)$ , Space:  $O(n)$ 
33
34 //Fungsi B
35 function groupAnagram(arr) {
36     let map = {};
37
38     for (let word of arr) {
39         let sorted = word.split('').sort().join('');
40
41         if (!map[sorted]) {
42             map[sorted] = [];
43         }
44         map[sorted].push(word);
45     }
46     return Object.values(map);
47 }
48 // Time:  $O(n * k \log k)$ ,  $n$  = jumlah kata,  $k$  = panjang kata, Space:  $O(n)$ 
```

```

50 //Fungsi C || Versi  $O(n^3)$  | Alasan: triple nested Loop || Time:  $O(n^3)$ , Space:  $O(1)$ 
51 function threeSumOn3(arr, target) {
52     let n = arr.length;
53     for (let i = 0; i < n; i++) {
54         for (let j = i + 1; j < n; j++) {
55             for (let k = j + 1; k < n; k++) {
56                 if (arr[i] + arr[j] + arr[k] === target) {
57                     return true;
58                 }
59             }
60         }
61     }
62     return false;
63 }
64 // Versi  $O(n^2)$  | Alasan: gunakan Set untuk cek pasangan || Time:  $O(n^2)$ , Space:  $O(n)$ 
65 function threeSumOn2(arr, target) {
66     let n = arr.length;
67     for (let i = 0; i < n; i++) {
68         let set = new Set();
69
70         for (let j = i + 1; j < n; j++) {
71             let needed = target - arr[i] - arr[j];
72             if (set.has(needed)) {
73                 return true;
74             }
75             set.add(arr[j]);
76         }
77     }
78     return false;
79 }
80 function hitungKompleksitas(n, fn, ...args) {
81     const start = Date.now();
82     const result = fn(...args);
83     const end = Date.now();
84     console.log("Hasil:", result);
85     console.log(`Waktu eksekusi: ${end - start} ms\n`);
86 }
87 console.log("=== Fungsi A ===");
88 hitungKompleksitas(0, intersectionOn2, [1,2,3,4], [3,4,5]);
89 hitungKompleksitas(0, intersectionOn, [1,2,3,4], [3,4,5]);
90 console.log("=== Fungsi B ===");
91 hitungKompleksitas(0, groupAnagram, ['eat','tea','tan','ate','nat','bat']);
92 console.log("=== Fungsi C ===");
93 hitungKompleksitas(0, threeSumOn3, [1,2,3,4,5], 9);
94 hitungKompleksitas(0, threeSumOn2, [1,2,3,4,5], 9);

```

```

PS E:\Praktikum Struktur Data\202573010034_strukturdatadanaalgoritma\praktikum-5> node .\tugas\tugas-1.js
=== Fungsi A ===
Hasil: [ 3, 4 ]
Waktu eksekusi: 0 ms

Hasil: [ 3, 4 ]
Waktu eksekusi: 0 ms

=== Fungsi B ===
Hasil: [ [ 'eat', 'tea', 'ate' ], [ 'tan', 'nat' ], [ 'bat' ] ]
Waktu eksekusi: 0 ms

=== Fungsi C ===
Hasil: true
Waktu eksekusi: 0 ms

Hasil: true
Waktu eksekusi: 1 ms

```


4. Program tugas 2 beserta hasil outputnya.

```

1 // O(1)
2 function fn_O1(n) {
3     return n + 1;
4 }
5
6 // O(n)
7 function fn_On(n) {
8     let sum = 0;
9     for (let i = 0; i < n; i++) {
10         sum += i;
11     }
12     return sum;
13 }
14
15 // O(n Log n)
16 function fn_OnLogn(n) {
17     let count = 0;
18
19     for (let i = 0; i < n; i++) {
20         for (let j = 1; j < n; j *= 2) {
21             count++;
22         }
23     }
24
25     return count;
26 }
27
28 // O(n²)
29 function fn_On2(n) {
30     let count = 0;
31
32     for (let i = 0; i < n; i++) {
33         for (let j = 0; j < n; j++) {
34             count++;
35         }
36     }
37
38     return count;
39 }
40
41 function benchmarkSemua(ukuranData) {
42     for (let n of ukuranData) {
43         console.log(`\n=== n = ${n} ===`);
44         let start, end;
45
46         // O(1)
47         start = Date.now();
48         fn_O1(n);
49         end = Date.now();
50         console.log(`O(1): ${end - start} ms`);
51
52         // O(n)
53         start = Date.now();
54         fn_On(n);
55         end = Date.now();
56         console.log(`O(n): ${end - start} ms`);
57
58         // O(n Log n)
59         start = Date.now();
60         fn_OnLogn(n);
61         end = Date.now();
62         console.log(`O(n log n): ${end - start} ms`);
63
64         // O(n²)
65         start = Date.now();
66         fn_On2(n);
67         end = Date.now();
68         console.log(`O(n²): ${end - start} ms`);
69     }
70 }
71 benchmarkSemua([100, 500, 1000, 5000, 10000]);
72
73 /*
74 Hasil:
75 - O(1): waktu konstan (tidak berubah signifikan saat n bertambah)
76 - O(n): bertambah linear
77 - O(n Log n): Lebih cepat dari O(n²) tapi Lebih Lambat dari O(n)
78 - O(n²): meningkat sangat drastis saat n besar
79 Kesimpulan:
80 Pola pertumbuhan waktu eksekusi sesuai dengan teori Big-O.
81 Semakin tinggi kompleksitas, semakin cepat waktu eksekusi meningkat.
82 */

```

```

PS E:\Praktikum Struktur Data\202573010034_strukturdatadanalgoritma\praktikum-5> node .\tugas\tugas-2.js

=== n = 100 ===
O(1): 0 ms
O(n): 0 ms
O(n log n): 0 ms
O(n²): 0 ms

=== n = 500 ===
O(1): 0 ms
O(n): 0 ms
O(n log n): 0 ms
O(n²): 2 ms

=== n = 1000 ===
O(1): 0 ms
O(n): 0 ms
O(n log n): 1 ms
O(n²): 3 ms

=== n = 5000 ===
O(1): 0 ms
O(n): 0 ms
O(n log n): 0 ms
O(n²): 51 ms

=== n = 10000 ===
O(1): 0 ms
O(n): 0 ms
O(n log n): 0 ms
O(n²): 185 ms

```

3.3 Submit ke GitHub

Setelah semua tugas, latihan, dan contoh selesai, lakukan push ke *repository* GitHub.

1. Ketik `cd ..` di Git Bush agar tidak berada dalam `/praktikum-5` tetapi di root folder repository. Karena Git diinisialisasi di folder utama (root)

```

newuser@DESKTOP-JP3BRBO MINGW64 /e/Praktikum Struktur Data/202573010034_strukturdatadanalgoritma/praktikum-5 (main)
$ cd ..

```

2. Ketik `git status` untuk cek status file

```

newuser@DESKTOP-JP3BRBO MINGW64 /e/Praktikum Struktur Data/202573010034_strukturdatadanalgoritma (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   praktikum-2/01-variabel.js
    modified:   praktikum-2/02-tipe-data.js
    modified:   praktikum-2/03-operator.js
    modified:   praktikum-2/04-kondisional.js
    modified:   praktikum-2/05-perulangan.js
    modified:   praktikum-2/tugas/tugas1.js
    modified:   praktikum-2/tugas/tugas2.js
    modified:   praktikum-3/tugas/tugas-2.js
    modified:   praktikum-4/02-destructuring-spread.js
    modified:   praktikum-4/03-class-dasar.js
    modified:   praktikum-4/04-inheritance.js
    modified:   praktikum-4/tugas/tugas-1.js
    modified:   praktikum-4/tugas/tugas-2.js

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    praktikum-5/

no changes added to commit (use "git add" and/or "git commit -a")

```

3. Lalu ketik `git add praktikum-5/` untuk menambahkan

```
newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_strukturdatadanalgoritma (main)
$ git add praktikum-5/
warning: in the working copy of 'praktikum-5/package.json', LF will be replaced by CRLF the next time Git touches it
```

4. Buat commit dengan pesan yang deskriptif. Seperti `git commit -m "Praktikum 5: Analisis Kompleksitas Algoritma"`.

```
newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_strukturdatadanalgoritma (main)
$ git commit -m "Praktikum 5: Analisis Kompleksitas Algoritma"
[main 95dabcf] Praktikum 5: Analisis Kompleksitas Algoritma
8 files changed, 505 insertions(+)
create mode 100644 praktikum-5/01-intro-kompleksitas.js
create mode 100644 praktikum-5/02-kelas-big-o.js
create mode 100644 praktikum-5/03-space-complexity.js
create mode 100644 praktikum-5/04-refactor.js
create mode 100644 praktikum-5/latihan-1.js
create mode 100644 praktikum-5/package.json
create mode 100644 praktikum-5/tugas/tugas-1.js
create mode 100644 praktikum-5/tugas/tugas-2.js
```

5. Lalu lakukan push ke repository GitHub.

```
newuser@DESKTOP-JP3BR80 MINGW64 /e/Praktikum Struktur Data/202573010034_strukturdatadanalgoritma (main)
$ git push origin main
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 8 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (12/12), 5.30 KiB | 1.06 MiB/s, done.
Total 12 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/faraa12/202573010034_strukturdatadanalgoritma.git
b3a7656..95dabcf main -> main
```

6. Hasil akhir



BAB IV PEMBAHASAN DAN ANALISIS

4.1 PEMBAHASAN

Percobaan pertama berfokus pada perbandingan algoritma dengan kompleksitas yang berbeda, yaitu $O(1)$, $O(n)$, dan $O(n^2)$. Algoritma dengan kompleksitas $O(1)$ menggunakan pendekatan rumus matematika sehingga hanya membutuhkan satu operasi независимо dari ukuran input. Sementara itu, algoritma $O(n)$ menggunakan perulangan tunggal yang menyebabkan waktu eksekusi meningkat seiring bertambahnya jumlah data. Adapun algoritma $O(n^2)$ menggunakan nested loop yang menyebabkan jumlah operasi meningkat secara signifikan.

Hasil percobaan menunjukkan bahwa algoritma $O(1)$ memiliki waktu eksekusi paling cepat, diikuti oleh $O(n)$, dan yang paling lambat adalah $O(n^2)$. Perbedaan ini semakin terlihat ketika ukuran data diperbesar. Hal ini membuktikan bahwa pemilihan algoritma sangat berpengaruh terhadap performa program.

Selain itu, dilakukan juga pengujian menggunakan fungsi `Date.now()` untuk mengukur waktu eksekusi secara empiris. Dari hasil tersebut terlihat bahwa algoritma dengan kompleksitas lebih tinggi membutuhkan waktu yang jauh lebih lama meskipun menghasilkan output yang sama.

Pada percobaan berikutnya, dilakukan analisis terhadap space complexity. Algoritma yang hanya menggunakan variabel sederhana memiliki kompleksitas ruang $O(1)$, sedangkan algoritma yang membuat struktur data baru seperti array memiliki kompleksitas $O(n)$. Selain itu, penggunaan rekursi juga meningkatkan penggunaan memori karena adanya call stack.

Selanjutnya, dilakukan refactoring terhadap algoritma yang memiliki kompleksitas tinggi. Algoritma dengan nested loop ($O(n^2)$) diubah menggunakan struktur data seperti Set atau Object sehingga kompleksitasnya menjadi $O(n)$. Hal ini menunjukkan bahwa penggunaan struktur data yang tepat dapat meningkatkan efisiensi secara signifikan.

4.2 ANALISA

Berdasarkan hasil praktikum yang telah dilakukan, dapat dianalisis bahwa kompleksitas algoritma memiliki peran yang sangat penting dalam menentukan performa

suatu program. Algoritma dengan kompleksitas rendah cenderung lebih efisien dan dapat menangani data dalam jumlah besar dengan lebih baik dibandingkan algoritma dengan kompleksitas tinggi.

Perbedaan kompleksitas seperti $O(n)$ dan $O(n^2)$ mungkin tidak terlalu terlihat pada data kecil, namun akan menjadi sangat signifikan ketika jumlah data meningkat. Hal ini sesuai dengan konsep Big O Notation yang menekankan pada laju pertumbuhan algoritma terhadap ukuran input, bukan waktu absolut.

Penggunaan metode empiris seperti `Date.now()` juga membantu dalam memahami performa algoritma secara nyata. Dengan pengujian langsung, mahasiswa dapat melihat bahwa teori yang dipelajari sesuai dengan hasil implementasi di lapangan.

Dari sisi penggunaan memori, terlihat bahwa terdapat hubungan antara waktu dan ruang (time-space trade-off). Algoritma yang lebih cepat sering kali membutuhkan memori tambahan, seperti penggunaan Set atau memoization. Oleh karena itu, pemilihan algoritma tidak hanya mempertimbangkan kecepatan, tetapi juga efisiensi penggunaan memori.

Selain itu, kemampuan dalam mengenali pola kompleksitas dari struktur kode menjadi keterampilan penting. Dengan memahami pola seperti loop tunggal, nested loop, dan rekursi, seorang programmer dapat dengan cepat memperkirakan kompleksitas suatu algoritma tanpa harus melakukan pengujian terlebih dahulu. Secara keseluruhan, praktikum ini memberikan pemahaman bahwa optimasi algoritma tidak selalu berarti membuat kode lebih rumit, melainkan memilih pendekatan yang paling efisien dan sesuai dengan kebutuhan.

BAB V PENUTUP

5.1 KESIMPULAN

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa analisis kompleksitas algoritma merupakan aspek yang sangat penting dalam pengembangan perangkat lunak, terutama dalam menentukan efisiensi suatu program. Dengan menggunakan notasi Big O, seorang programmer dapat memahami bagaimana performa algoritma akan berubah seiring dengan bertambahnya ukuran data yang diproses.

Melalui praktikum ini, terlihat bahwa algoritma dengan kompleksitas rendah seperti $O(1)$ dan $O(n)$ memiliki performa yang jauh lebih baik dibandingkan algoritma dengan kompleksitas tinggi seperti $O(n^2)$ atau $O(2^n)$, terutama ketika digunakan pada data dalam jumlah besar. Selain itu, penggunaan teknik optimasi seperti pemilihan struktur data yang tepat juga terbukti mampu meningkatkan efisiensi algoritma secara signifikan.

Tidak hanya dari sisi waktu, penggunaan memori juga menjadi faktor penting yang harus diperhatikan. Oleh karena itu, pemahaman terhadap konsep time complexity dan space complexity secara bersamaan sangat diperlukan agar dapat menghasilkan solusi yang optimal. Dengan demikian, mahasiswa diharapkan mampu menerapkan konsep-konsep ini dalam pembuatan program yang efisien, efektif, dan sesuai dengan kebutuhan di dunia nyata.

BAB VI DAFTAR PUSAKA

Zulman, M. Reza. (2026). *Modul Praktikum Struktur Data dan Algoritma: Analisis Kompleksitas Algoritma*. Politeknik Negeri Lhokseumawe.

MDN Web Docs. (2025). *JavaScript Performance*.

W3Schools. (2025). *JavaScript Performance and Optimization*.